

## Background

When dealing with large CFD datasets, running ParaView interactively on a local machine can become painfully slow or even infeasible due to memory and processing limits. This becomes even more difficult for remote users accessing a cluster, as downloading the CFD datasets adds another layer of complexity. The **pvpython** module, which is ParaView's Python scripting interface, can provide the full ParaView server-side API. It lets you automate ParaView tasks such as loading data, applying filters, and saving images or extracted data – all **without launching the GUI**.

## How pvpython helps with heavy CFD workloads?

### 1. Headless (non-GUI) operation

- pvpython runs scripts without the ParaView GUI, reducing memory overhead and avoiding GUI-related crashes with big datasets.
- You can run pvpython on remote high-performance computing (HPC) clusters where GUI access may not even be possible.

### 2. Automation of repetitive tasks

- You can automate repetitive workflows: batch process multiple time steps, apply filters, export image/CSV files, generate plots, etc.
- This is useful for parametric studies or ensemble runs, where you might have hundreds of cases.

### 3. Parallel processing

- pvpython can launch in parallel using `mpirun -np N pvpython my_script.py` to process data using multiple cores.
- It can significantly speed up operations like contouring, slicing, thresholding, and exporting.

## Typical workflow

Here's an example of a pvpython workflow as below:

```
python:

from paraview.simple import *

# Load heavy CFD data
data = OpenDataFile("/path/to/large/cfd/solution.vtk")

# Apply filters
contour = Contour(Input=data)
contour.ContourBy = ['POINTS', 'pressure']
contour.Isosurfaces = [1e5]

# Save output
SaveData("/output/contour.vtp", proxy=contour)
```

You can run this as:

```
bash:

pvpython my_script.py
```

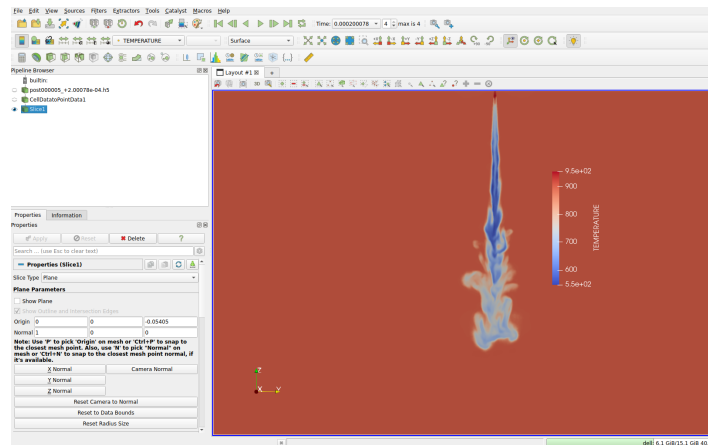
## ECN Spray-A flame image processing

This example demonstrates how to generate slices of the ECN Spray-A flame simulation performed using CONVERGE. The CFD results are stored in HDF5 (.h5) format, which is broadly supported by most versions of ParaView. In this case, multiple .h5 files represent different time steps, and the Python script is designed to produce a sequence of flame images for the specified range of post-processing files, based on the defined start and end times.

```
sk10712@dell:/media/sk10712/One Touch1/NYUAD/SJU-NYU/PVpython.example/output$
427008
kr-x 2 sk10712 sk10712 262144 Jun 29 15:18 ./
kr-x 7 sk10712 sk10712 262144 Jun 29 15:19 ../
kr-x 1 sk10712 sk10712 42127652 Jun 26 10:03 post000001_+0.00000e+00.h5*
kr-x 1 sk10712 sk10712 49727740 Jun 26 10:03 post000002_+5.00535e-05.h5*
kr-x 1 sk10712 sk10712 76723368 Jun 26 10:06 post000003_+1.00085e-04.h5*
kr-x 1 sk10712 sk10712 114022184 Jun 26 10:08 post000004_+1.50000e-04.h5*
kr-x 1 sk10712 sk10712 153390196 Jun 26 10:10 post000005_+2.00078e-04.h5*
```

## Generate .pvsm

The first step is to generate a slice image from a randomly selected post-processing file on your local machine. This is a crucial step, as it allows you to visually verify the intended output using the ParaView GUI. To do this, make sure you've downloaded at least one post file from the HPC that is small enough to be processed locally.



Normally, outside of the pvpython workflow, you might capture a snapshot of the image manually using the “File > Save Screenshot” option in the ParaView GUI, repeating this process for each post file in the dataset.

To streamline this process, the next step involves creating a .pvsm file—ParaView’s state file—which captures the full session setup. This lets you reopen the session later with all filters and settings preserved, enabling consistent reproduction of the visualization environment. You can do this by selecting “File > Save State” in the ParaView GUI. The image below shows a part of the contents from the resulting .pvsm file.

```
<ParaView>
<ServerManagerState version="5.11.0">
  <Proxy group="animation" type="AnimationScene" id="263" servers="16">
    <Property name="AnimationTime" id="263.AnimationTime" number_of_elements="1">
      <Element index="0" value="0.00020007847149644369"/>
    </Property>
    <Property name="Cues" id="263.Cues" number_of_elements="1">
      <Proxy value="265"/>
      <Domain name="groups" id="263.Cues.groups"/>
    </Property>
    <Property name="Duration" id="263.Duration" number_of_elements="1">
      <Element index="0" value="10"/>
    </Property>
    <Property name="EndTime" id="263.EndTime" number_of_elements="1">
      <Element index="0" value="0.00020007847149644369"/>
    </Property>
    <Property name="FramesPerTimestep" id="263.FramesPerTimestep" number_of_elements="1">
      <Element index="0" value="1"/>
      <Domain name="range" id="263.FramesPerTimestep.range"/>
    </Property>
  </Proxy>
</ServerManagerState>
```

## Generate a python script: Start Trace / Stop Trace in the ParaView GUI

The next step involves creating a python script in a format interpretable by the pvpython interface. The script will replicate the operations you would typically carry out in the ParaView GUI, such as loading a .pvsm state file and using the “Save Screenshot” function – effectively automating the process outside the standard GUI workflow.

To begin, reopen ParaView and select “Tools > Start Trace”, then click “OK” in the pop-up window. This feature records your actions in the GUI as a Python script, which can later be executed using the pvpython module.

Next, open the previously saved .pvsm file by selecting “File > Load State” and clicking “OK”. Then, go to “File > Save Screenshot” to capture and save an image. Keep in mind that this step is meant to produce a Python script that will later be adapted to handle multiple post-processing files. So, the specific post file you use at this stage is not important.

Finally, complete the process by selecting “Tools > Stop Trace”. A new window will appear displaying the generated Python script. Save this script, as it will be used in the next step.

```
# trace generated using paraview version 5.11.0
#import paraview
#paraview.compatibility.major = 5
#paraview.compatibility.minor = 11

### import the simple module from the paraview
from paraview.simple import *
### disable automatic camera reset on 'Show'
paraview.simple._DisableFirstRenderCameraReset()

# get active view
renderView1 = GetActiveViewOrCreate('RenderView')

# destroy renderView1
Delete(renderView1)
del renderView1

# get animation scene
animationScene1 = GetAnimationScene()

# get the time-keeper
timeKeeper1 = GetTimeKeeper()

# load state
LoadState('/media/sk10712/One Touch1/NYUAD/SJU-NYU/PVpython.example/output/temperature.slice.pvsm')

# find view
renderView1 = FindViewOrCreate('RenderView1', viewtypes='RenderView')

# set active view
SetActiveView(renderView1)

# get layout
layout1 = GetLayout()

# layout/tab size in pixels
layout1.SetSize(1374, 789)

# current camera placement for renderView1
renderView1.CameraPosition = [-0.1801511549381651, -0.0016111313076591707, -0.0477519397736826]
renderView1.CameraFocalPoint = [0.1150469662106417, -0.0016111313076591707, -0.0477519397736826]
renderView1.CameraViewUp = [0.0, 0.0, 1.0]
renderView1.CameraParallelScale = 0.07640289583179231

# save screenshot
saveScreenshot('/media/sk10712/One Touch1/NYUAD/SJU-NYU/PVpython.example/output/temperature.png', renderView1, ImageResolution=[1374, 789])
```

If you have pvpython available on your local machine, you can run this script by running following command in your bash:

*bash:*

pvpython my\_python\_script.py

## Modify the script for automated image processing

The next step is to adjust the Python script created in the previous step, so it can be executed over multiple post-processing .h5 files. To achieve this, we'll organize the script into a subroutine named *readPVSM* in this example and it can be invoked by a separate main function. The subroutine function includes the modified Python script that was generated in the previous step. The main function is written as shown below in the Python format.

```
pvsM = './temperature.slice.pvsM'
filePath = './output'
filePrefix = 'post'
pltPrefix = 'temperature'
# create 'figures' folder
figurepath = os.path.join('figures')
try:
    os.mkdir(figurepath)
    message = "Directory " + figurepath + " Created."
    print(message)
except:
    message = "Directory " + figurepath + " already exists."

timeList, numList, postFileList = sortH5FileListInTime(filePath, filePrefix)
tStart = 0.0
tEnd = 5.0e-03
for kk, tt in enumerate(timeList):
    if tt < tStart or tt > tEnd:
        continue
    num = "%06d" % numList[kk]
    postf = postFileList[kk]
    print("| Processing", postf, "...")

    pngFile = pltPrefix + str(num) + '_%.5E.png' % tt
    pngFile = os.path.join(figurepath, pngFile)
    readPVSM(tt, pltPrefix, num, pvsM, postf, pngFile)
gc.collect()
```

The *readPVSM* subroutine essentially incorporates the Python script generated in the previous step, with a few modification to make it executable within the main routine. You can directly paste the script's content into this subroutine. Note that the original .pvsM file was designed for the randomly selected post file, the first few lines in this subroutine are designed to update the post file name, time, and other relevant parameters accordingly as highlighted in the image below:

```
def readPVSM(time, pltPrefix, num, pvsM, postf, pngfile):
    # postfile name without output path
    pf = postf.split('/')[1]

    pvsMnew = pltPrefix + str(num) + ' %.5E.pvsM' % time
    checkWords = ([postFile], [currentTime], [posth5])
    repWords = (postf, str(time), pf)
    with open(pvsM, 'rt') as fin:
        with open(pvsMnew, 'wt') as fout:
            for line in fin:
                for check, rep in zip(checkWords, repWords):
                    line = line.replace(check, rep)
                fout.write(line)

    ##### disable automatic camera reset on 'Show'
    paraview.simple._DisableFirstRenderCameraReset()

    # get active view
    renderView1 = GetActiveViewOrCreate('RenderView')

    # get animation scene
    animationScene1 = GetAnimationScene()

    # get the time-keeper
    timeKeeper1 = GetTimeKeeper()

    # load state
    LoadState(pvsMnew)

    # find view
    renderView1 = FindViewOrCreate('RenderView1', viewType='RenderView')

    # set active view
    SetActiveView(renderView1)

    # get layout
    layout1 = GetLayout()

    # layout/tab size in pixels
    layout1.SetSize(1374, 789)

    # current camera placement for renderView1
    renderView1.CameraPosition = [-0.1801511549381651, -0.0016111313076591707, -0.0477519397736826]
    renderView1.CameraFocalPoint = [0.1150469662106417, -0.0016111313076591707, -0.0477519397736826]
    renderView1.CameraViewUp = [0.0, 0.0, 1.0]
    renderView1.CameraParallelScale = 0.07640289583179231

    # save screenshot
    SaveScreenshot(pngfile, renderView1, ImageResolution=[1374, 789])
    print(pngfile, "created.")
```

To execute this update, the following strings should be identified in the .pvsM file. Please refer to the example scripts and .pvsM file.

### Run your pvpython script on HPC (running on interactive session)

If pvpython is available in your HPC environment, you can execute this script using the following bash command, just as you would on your local machine.

```
bash:
```

```
pvpython my_python_script.py
```

However, it's not advisable to run this task on the login nodes, as doing so violates NYU cluster policies. More importantly, you are likely to face memory issues due to the limited memory allocated to general users. To prevent this issue, it is strongly recommended to execute the Python script within an interactive session, as clearly outlined in the Jubail documentation webpage: <https://crc-docs.abudhabi.nyu.edu/hpc/jobs/interactive.html>

You can initiate an interactive session using the command below. In this example, one CPU core is requested on a compute node with 256 GB of allocated memory.

```
bash:
```

```
srun -pty -mem 256G -t 05:00:00 -n 1 /bin/bash
```

```
(py371) hs6115@login2:~$ srun --pty --mem 256G -t 5:00:00 -n 1 /bin/bash
srun: job 11056840 queued and waiting for resources
srun: job 11056840 has been allocated resources
Unloading module 'gcc/9.2.0'
Loading module 'gcc/9.2.0'
(py371) hs6115@cn148:~$
```